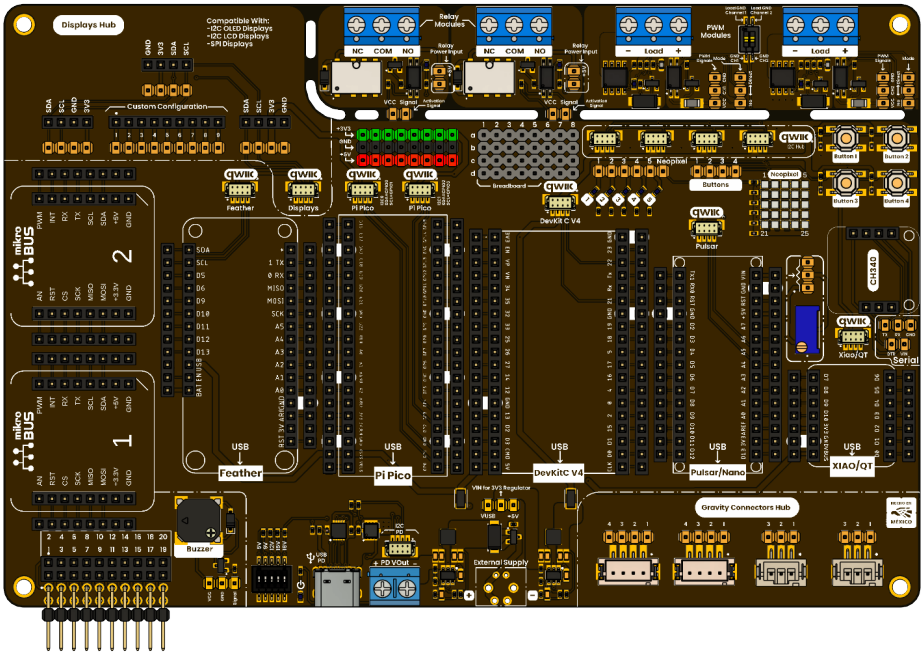

DevLab: Multi Hub Shield User Guide and Technical Reference

Release 0.0.1

Department of Research, Innovation, and Development

Mar 02, 2026

General Information

1 Terms, Acknowledgments, and Licenses **3**
1.1 Terms and Conditions . . . 3
1.2 Acknowledgments and Contributors . . . 3
1.3 Hardware License . . . 3
1.4 Documentation License . . . 4
1.5 Software and Firmware License . . . 4
1.6 Resources and References . . . 4
1.7 Ecosystem Standards and References . . . 4
1.8 Library References . . . 4
1.9 Disclaimer . . . 5
1.10 Support and Community . . . 5
1.11 Revision History . . . 5
1.12 Copyright Notice . . . 5

2 General Information **7**
2.1 Supported Microcontroller Platforms . . . 7
2.2 Supported Interfaces and Features . . . 7
2.3 Power Options . . . 8
2.4 DevLab Ecosystem Integration . . . 8
2.5 Relay Modules . . . 8
2.6 PWM Interfaces . . . 8
2.7 Display Options . . . 8
2.8 Gravity Connectors . . . 8
2.9 Expansion Options . . . 9
2.10 Board Specifications . . . 9

3 Getting Started **11**
3.1 Hardware Setup . . . 11
3.2 Software Setup . . . 11
3.3 Quick Test - LED Blink . . . 12
3.4 Testing Peripherals . . . 12
3.5 I2C Devices (QWIIC/STEMMA) . . . 12
3.6 OLED Display . . . 13
3.7 Relay Control . . . 13
3.8 Next Steps . . . 14
3.9 Troubleshooting . . . 14

4 Platform Compatibility **15**
4.1 Overview . . . 15
4.2 Supported Platforms . . . 15
4.3 Pin Mapping and Configuration . . . 17
4.4 Software Support . . . 18
4.5 Platform Selection Guide . . . 18

5 Connectivity Interfaces **19**
5.1 Overview . . . 19
5.2 I2C Interface . . . 19
5.3 SPI Interface . . . 20
5.4 UART Interface . . . 20
5.5 GPIO (General Purpose Input/Output) . . . 21
5.6 ADC (Analog to Digital Converter) . . . 21
5.7 Interface Selection Guide . . . 22
5.8 Best Practices . . . 22

6 LED Matrix **25**
6.1 Overview . . . 25
6.2 Technical Specifications . . . 25
6.3 Features . . . 25
6.4 Pin Configuration . . . 25
6.5 Programming Examples . . . 25
6.6 Animation Examples . . . 26
6.7 Practical Applications . . . 28
6.8 Best Practices . . . 28
6.9 Power Consumption . . . 28
6.10 Troubleshooting . . . 29

7 Display Hub - OLED Display **31**
7.1 Overview . . . 31
7.2 Technical Specifications . . . 31
7.3 Display Features . . . 31
7.4 Pin Connections . . . 31
7.5 Programming Libraries . . . 31
7.6 Basic Examples . . . 32
7.7 Graphics Examples . . . 32
7.8 Sensor Data Display . . . 33
7.9 Advanced Features . . . 34
7.10 MicroPython Example . . . 35
7.11 TFT Display Configuration . . . 35
7.12 Best Practices . . . 35
7.13 Troubleshooting . . . 35
7.14 Applications . . . 36

8 devlab ecosystem integration **37**
8.1 devlab concept . . . 37
8.2 devlab connector . . . 37
8.3 design philosophy . . . 38
8.4 official identification . . . 38
8.5 qwiic/stemma qt compatibility . . . 38
8.6 technical specifications . . . 39
8.7 pinout . . . 39
8.8 Compatible Modules . . . 39
8.9 Daisy-Chaining . . . 40
8.10 Programming Examples . . . 40
8.11 Best Practices . . . 42
8.12 Common Issues and Solutions . . . 42
8.13 Project Ideas . . . 42
8.14 Ecosystem Resources . . . 42

9 Relay Modules **45**

9.1 Overview . . . 45
9.2 Technical Specifications . . . 45
9.3 Relay Construction . . . 45
9.4 Control Interface . . . 45
9.5 Programming Examples . . . 46
9.6 Wiring Examples . . . 47
9.7 Safety Considerations . . . 47
9.8 Practical Applications . . . 48
9.9 Advanced Features . . . 48
9.10 Troubleshooting . . . 49
9.11 Specifications Summary . . . 49

10 How to Generate an Error Report **51**
10.1 Steps to Create an Error Report . . . 51
10.2 Review and Follow-Up . . . 51

Note: This documentation is actively evolving. For the latest updates and revisions, please visit the project's GitHub repository at https://github.com/UNIT-Electronics-MX/devlab_multi_hub_shield

Introduction

The **DevLab Multi Hub Shield** is a modular shield expansion board designed to be compatible with various microcontroller platforms, including **Pulsar**, **Xiao**, **Pi Pico**, **Feather**, **Devkit V4**, and **Mikro Bus**. The shield provides a wide range of interfaces and features for prototyping and development, making it an ideal choice for makers, educators, and developers looking to expand their projects with additional connectivity and functionality.

Key Features

Table 1: Overview

Feature	Description
Support Shield Format	Pulsar, Xiao, Pi Pico, Feather, Devkit V4, Mikro Bus
Connectivity	I2C, SPI, UART, GPIO, ADC
LED Matrix	5x5 RGB LED matrix for visual feedback
DevLab Ecosystem	7 Compatible connectors (JST 1mm) with QWIIC-STEMMA compatibility
Relay Modules	2 Relay Modules
Display Hub	I2C 128x64 OLED Display - Display customization configuration TFT
PWM Interface	2 PWM Modules
Potentiometer	1 Potentiometer for analog input
Gravity Interface	4 Gravity connectors for sensor integration (2x4 and 2x3 Connectors)
PWM Buzzer	1 PWM Buzzer for audio feedback
Expansion Lines	4 Breadboard lines for custom circuit connections
Expansion Connectors	1 connector with 20 pins for additional modules and shields
Buttons	4 Buttons for user input and control
LEDs	4 LEDs for status indication and feedback
Power Delivery	Supports USB-C, 3.3V, 5V, and external power supply options for versatile power management

Advantages

- **Multi-platform compatibility**: Works with various microcontroller form factors
- **Rich peripheral set**: Includes relays, sensors, displays, and expansion ports
- **DevLab ecosystem integration**: Seamless connectivity with QWIIC-STEMMA compatible modules
- **Educational friendly**: Perfect for learning and teaching embedded systems
- **Prototyping ready**: Multiple expansion options for custom circuits
- **Flexible power options**: Multiple power input methods for different scenarios

Applications

- **Prototyping**: Quickly develop and test ideas with multiple interface options
- **Education**: Suitable for learning microcontroller basics and advanced concepts
- **IoT Projects**: Build connected devices with multiple sensor integration points
- **Automation**: Control relays and actuators for home and industrial automation
- **Display Projects**: Create visual interfaces with LED matrix and OLED display
- **Sensor Integration**: Easy connection to multiple sensors via Gravity and QWIIC connectors

Compatible Platforms

The DevLab Multi Hub Shield supports the following microcontroller platforms:

- **Pulsar**: UNIT Electronics' Pulsar development board
- **Seeed Xiao**: Xiao ESP32, Xiao RP2040, and other Xiao variants
- **Raspberry Pi Pico**: RP2040-based boards
- **Adafruit Feather**: Various Feather-format boards
- **ESP32 Devkit V4**: ESP32 development boards
- **Mikro Bus**: MikroElektronika Click boards ecosystem

Resources

- GitHub Repository: UNIT-Electronics-MX/devlab_multi_hub_shield
- Hardware Files: Available in the repository's `hardware/` directory

- Example Code: Available in the repository's `software/examples/` directory
- Community Support: GitHub issues and UNIT Electronics forums
- Shop: UNIT Electronics Store

CHAPTER
ONE

TERMS, ACKNOWLEDGMENTS, AND LICENSES

1.1 Terms and Conditions

By using, modifying, or distributing the documentation, firmware, or hardware designs included in this repository, you agree to the following terms:

- All materials are provided **"as-is"**, without warranty of any kind.
- The authors and contributors shall not be held responsible for **any damages**, data loss, or legal issues arising from the use of these materials.
- Usage is intended for **educational, development, prototyping**, and other lawful purposes.
- When redistributing or reusing any part of this project, you must **retain attribution** and comply with the corresponding license terms of each component.

1.2 Acknowledgments and Contributors

This project is developed and maintained by:

UNIT Electronics

- **Website**: https://uelectronics.com/
- **GitHub**: https://github.com/UNIT-Electronics-MX
- **Project Repository**: DevLab Multi Hub Shield

1.2.1 Community Contributions

We welcome contributions from the community. Special thanks to:

- All users who report issues and suggest improvements
- Contributors who submit pull requests and documentation improvements
- The maker and developer community for testing and feedback

1.2.2 Third-Party Components

The DevLab Multi Hub Shield integrates with various ecosystems and standards:

- **QWIIC/STEMMA QT**: SparkFun and Adafruit connector standard for I2C devices
- **Gravity**: DFRobot sensor connector standard
- **Arduino Libraries**: Various open-source libraries for displays, sensors, and peripherals
- **MicroPython**: Python implementation for microcontrollers

1.3 Hardware License

All hardware designs (schematics, layouts, PCB files, and design documents) in this repository are released under the **MIT License**.

MIT License Summary:

- Commercial use permitted
- Modification permitted
- Distribution permitted
- Private use permitted
- Must include license and copyright notice
- No warranty provided

1.4 Documentation License

This user guide, documentation, and visual content are licensed under:

Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0)

CC BY-SA 4.0 Summary:

- Share — copy and redistribute the material
- Adapt — remix, transform, and build upon the material
- Attribution — must give appropriate credit
- ShareAlike — must distribute under same license
- Commercial use permitted

1.5 Software and Firmware License

Example code, firmware samples, and software utilities included in this repository are released under the **MIT License**, unless otherwise specified.

1.6 Resources and References

Table 1.1: Resources

Resource	URL
Project Repository	https://github.com/UNIT-Electronics-MX/devlab_multi_hub_shield
UNIT Electronics	https://uelectronics.com/
Documentation	GitHub Pages (this site)
Issues & Support	GitHub Issues

1.7 Ecosystem Standards and References

Table 1.2: Standards and Ecosystems

Standard/Platform	Reference
QWIIC (SparkFun)	https://www.sparkfun.com/qwiic
STEMMA QT (Adafruit)	https://www.adafruit.com/category/620
Gravity (DFRobot)	https://www.dfrobot.com/gravity
Arduino Platform	https://www.arduino.cc/
MicroPython	https://micropython.org/
Raspberry Pi Pico SDK	https://github.com/raspberrypi/pico-sdk
ESP-IDF (ESP32)	https://github.com/espressif/esp-idf

1.8 Library References

Common libraries used with the DevLab Multi Hub Shield:

Table 1.3: Common Libraries

Library	Purpose	Platform
Adafruit SSD1306	OLED Display driver	Arduino
Adafruit GFX	Graphics library	Arduino
FastLED	RGB LED control	Arduino
Wire	I2C communication	Arduino
SPI	SPI communication	Arduino
machine (MicroPython)	Hardware control	MicroPython
ssd1306 (MicroPython)	OLED Display	MicroPython

1.9 Disclaimer

Safety Notice

- The relay modules can switch high voltage and current. Always follow electrical safety guidelines.
- Ensure proper voltage levels when connecting external modules and peripherals.
- Do not exceed the maximum ratings of any component.
- Use appropriate fuses and protection circuits in your applications.

Compliance

- Users are responsible for ensuring their projects comply with local regulations and standards.
- Electromagnetic compatibility (EMC) and safety certifications are the responsibility of the end user.
- This product is intended for development and prototyping purposes.

No Warranty

THIS PRODUCT AND DOCUMENTATION ARE PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES, OR OTHER LIABILITY ARISING FROM THE USE OF THIS PRODUCT.

1.10 Support and Community

For support, questions, and discussions:

- **GitHub Issues**: Report bugs and request features at GitHub Issues
- **UNIT Electronics**: Contact through uelectronics.com
- **Community Forums**: Join discussions with other users and developers

1.11 Revision History

- **Version 1.0.0** - Initial release of DevLab Multi Hub Shield
- **Documentation**: Updated February 2026

1.12 Copyright Notice

Copyright © 2026 UNIT Electronics. All rights reserved.

Hardware designs, documentation, and example code are released under open-source licenses as specified above. The UNIT Electronics name and logo are trademarks of UNIT Electronics.

CHAPTER
TWO

GENERAL INFORMATION

The **DevLab Multi Hub Shield** is a modular shield expansion board designed to be compatible with various microcontroller platforms. It provides a comprehensive set of interfaces and features that make it ideal for prototyping, education, IoT projects, and automation applications.

2.1 Supported Microcontroller Platforms

The shield is compatible with the following development board formats:

- **Pulsar** — UNIT Electronics flagship development board
- **Seeed Xiao** — Compact form factor (ESP32, RP2040, SAMD21, etc.)
- **Raspberry Pi Pico** — RP2040-based boards
- **Adafruit Feather** — Feather-compatible boards
- **ESP32 Devkit V4** — Standard ESP32 development boards
- **Mikro Bus** — MikroElektronika Click boards ecosystem

All platforms can be connected via dedicated pin headers that provide access to power, GPIO, I2C, SPI, UART, and ADC interfaces.

2.2 Supported Interfaces and Features

The DevLab Multi Hub Shield integrates multiple interfaces and peripherals for versatile application development:

Table 2.1: Interface and Feature Overview

Interface/Featur	Description
I2C	Standard I2C bus for sensor and module communication
SPI	High-speed serial peripheral interface
UART	Serial communication interface
GPIO	General purpose input/output pins
ADC	Analog-to-digital converter inputs via potentiometer
LED Matrix	5x5 RGB LED matrix for visual feedback and displays
OLED Display	I2C 128x64 OLED display with TFT configuration options
Relay Modules	2 relay modules for switching applications
PWM Outputs	2 PWM modules for motor control and LED dimming
Buzzer	PWM-driven buzzer for audio feedback
Buttons	4 tactile buttons for user input
Status LEDs	4 indicator LEDs for feedback
DevLab Connectors	7 QWIIC-STEMMA compatible connectors (JST 1mm pitch)
Gravity Interface	4 Gravity connectors (2x4 and 2x3 pin configurations)
Breadboard Lines	4 breadboard-compatible expansion lines
Expansion Connector	20-pin header for additional modules and custom shields

2.3 Power Options

The DevLab Multi Hub Shield supports multiple power input methods:

- **USB-C**: Direct power via USB-C connector (5V)
- **3.3V Rail**: Low-voltage power supply for compatible modules
- **5V Rail**: Standard voltage for most peripherals
- **External Supply**: Dedicated input for external power sources

Note: Power distribution is managed through on-board voltage regulators. Ensure proper voltage selection when connecting external modules.

2.4 DevLab Ecosystem Integration

The shield features **7 QWIIC-STEMMA compatible connectors** (JST 1mm pitch) that enable seamless integration with the DevLab ecosystem and third-party I2C modules. These connectors support:

- Daisy-chaining multiple sensors and modules
- Hot-plugging capability
- Standardized pinout (VCC, GND, SDA, SCL)
- Compatibility with SparkFun QWIIC and Adafruit STEMMA QT systems

2.5 Relay Modules

Two integrated relay modules provide:

- Switching capability for AC/DC loads
- Isolated control signals
- LED indicators for relay status
- Screw terminal connections

2.6 PWM Interfaces

Two dedicated PWM outputs allow:

- Motor speed control
- Servo positioning
- LED brightness control
- Variable frequency signal generation

2.7 Display Options

LED Matrix (5x5 RGB)

- Individual addressable RGB LEDs
- Visual feedback and simple graphics
- Animation capabilities
- Programmable via GPIO

OLED Display (128x64)

- I2C interface
- High contrast ratio
- Text and graphics support
- TFT configuration options for expanded functionality

2.8 Gravity Connectors

Four Gravity-compatible connectors (DFRobot standard) provide:

- **2x4 pin connectors** (2 units): For digital/analog sensors
- **2x3 pin connectors** (2 units): For compact sensor modules
- Color-coded for easy identification
- Wide sensor compatibility

2.9 Expansion Options

Breadboard Lines

4 breadboard-compatible lines with standard 2.54mm pitch for:

- Custom circuit prototyping
- Temporary connections
- Wire jumper access to signals

20-Pin Expansion Connector

High-density connector providing:

- Access to all major bus signals
- Power distribution
- GPIO expansion
- Stacking capability for additional shields

2.10 Board Specifications

Table 2.2: Technical Specifications

Parameter	Value
Compatible Formats	Pulsar, Xiao, Pi Pico, Feather, Devkit V4, Mikro Bus
Operating Voltage	3.3V / 5V (selectable)
Input Power	USB-C 5V, External 3.3V-5V
I2C Connectors	7x QWIIC-STEMMA (JST 1mm)
Gravity Connectors	4x (2x4 pins + 2x3 pins)
Relays	2x electromechanical relays
PWM Outputs	2 channels
LEDs	4 status LEDs + 5x5 RGB matrix
Buttons	4 tactile push buttons
Display	128x64 I2C OLED
Expansion	20-pin header + 4 breadboard lines

CHAPTER
THREE

GETTING STARTED

This guide will help you set up and start using the **DevLab Multi Hub Shield** with your preferred microcontroller platform.

3.1 Hardware Setup

Step 1: Select Your Microcontroller Platform

The DevLab Multi Hub Shield is compatible with multiple platforms. Choose one of the following:

- Pulsar (UNIT Electronics)
- Seeed Xiao (ESP32, RP2040, SAMD21, etc.)
- Raspberry Pi Pico
- Adafruit Feather
- ESP32 Devkit V4
- Mikro Bus compatible boards

Step 2: Mount the Shield

1. Align the pin headers of your microcontroller board with the corresponding socket on the shield
2. Gently press the boards together ensuring all pins are properly seated
3. Verify that the boards are securely connected

Step 3: Power Connection

The shield can be powered through multiple methods:

- **USB-C**: Connect a USB-C cable to the shield's USB-C port (5V)
- **Microcontroller Power**: Power is shared from the microcontroller board
- **External Supply**: Use dedicated power input (check voltage requirements)

> **Warning:** Do not power the shield and microcontroller from multiple sources simultaneously unless you understand the power distribution design.

Step 4: Verify Power

Check the power indicator LEDs on the shield. If lit, the shield is receiving power correctly.

3.2 Software Setup

Arduino IDE Configuration

For Arduino-compatible boards (Xiao, Pico, ESP32):

1. Install the Arduino IDE from arduino.cc
2. Add your board's support package:
   - **Xiao ESP32**: Install "esp32" boards via Board Manager
   - **Raspberry Pi Pico**: Install "Arduino Mbed OS RP2040" or "Raspberry Pi Pico/RP2040"
   - **ESP32 Devkit**: Install "esp32" boards
   - **Pulsar**: Follow UNIT Electronics documentation
3. Select your board from **Tools** → **Board**
4. Select the correct port from **Tools** → **Port**

MicroPython Setup

For boards supporting MicroPython (Pico, ESP32):

1. Flash MicroPython firmware to your board
2. Use Thonny IDE or similar tool to connect
3. Upload example scripts from the `software/examples/mp/` directory

3.3 Quick Test - LED Blink

Let's verify your setup by blinking one of the status LEDs.

Arduino Code

```
// Define LED pin (check your shield␣
↪pinout)
#define LED_PIN 13

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(500);
  digitalWrite(LED_PIN, LOW);
  delay(500);
}
```

MicroPython Code

```
from machine import Pin
import time

# Define LED pin (check your shield pinout)
led = Pin(13, Pin.OUT)

while True:
    led.on()
    time.sleep(0.5)
    led.off()
    time.sleep(0.5)
```

3.4 Testing Peripherals

Buttons

The shield includes 4 buttons for user input. Test them with:

```
#define BUTTON_PIN 5  // Adjust to your␣
↪button pin

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
  if (digitalRead(BUTTON_PIN) == LOW) {
```

(continues on next page)

(continued from previous page)

```
    Serial.println("Button pressed!");
    delay(200);  // Simple debounce
  }
}
```

Buzzer

Test the PWM buzzer:

```
#define BUZZER_PIN 9  // Adjust to your␣
↪buzzer pin

void setup() {
  pinMode(BUZZER_PIN, OUTPUT);
}

void loop() {
  tone(BUZZER_PIN, 1000);  // 1kHz tone
  delay(500);
  noTone(BUZZER_PIN);
  delay(500);
}
```

Potentiometer (ADC)

Read analog values:

```
#define POT_PIN A0  // Adjust to your␣
↪potentiometer pin

void setup() {
  Serial.begin(115200);
}

void loop() {
  int value = analogRead(POT_PIN);
  Serial.print("Potentiometer value: ");
  Serial.println(value);
  delay(100);
}
```

3.5 I2C Devices (QWIIC/STEMMA)

The shield provides 7 QWIIC-STEMMA compatible connectors for I2C devices.

I2C Scanner Example

```
#include <Wire.h>
```

(continues on next page)

(continued from previous page)

```
void setup() {
  Wire.begin();
  Serial.begin(115200);
  Serial.println("I2C Scanner");
}

void loop() {
  byte error, address;
  int nDevices = 0;

  Serial.println("Scanning...");

  for(address = 1; address < 127;␣
↪address++ ) {
    Wire.beginTransmission(address);
    error = Wire.endTransmission();

    if (error == 0) {
      Serial.print("I2C device found at␣
↪address 0x");
      if (address<16) Serial.print("0");
      Serial.println(address, HEX);
      nDevices++;
    }
  }

  if (nDevices == 0)
    Serial.println("No I2C devices found");
  else
    Serial.println("Scan complete");

  delay(5000);
}
```

3.6 OLED Display

The shield includes a 128x64 I2C OLED display.

Required Library

Install the Adafruit SSD1306 library via Arduino Library Manager.

Example Code

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
```

(continues on next page)

(continued from previous page)

```
#define OLED_RESET -1

Adafruit_SSD1306 display(SCREEN_WIDTH,␣
↪SCREEN_HEIGHT, &Wire, OLED_RESET);

void setup() {
  if(!display.begin(SSD1306_SWITCHCAPVCC,␣
↪0x3C)) {
    Serial.println("SSD1306 allocation␣
↪failed");
    for(;;);
  }

  display.clearDisplay();
  display.setTextSize(2);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0,0);
  display.println("DevLab");
  display.println("Multi Hub");
  display.display();
}

void loop() {
  // Your code here
}
```

3.7 Relay Control

Control the 2 relay modules:

```
#define RELAY1_PIN 7  // Adjust to your␣
↪relay pins
#define RELAY2_PIN 8

void setup() {
  pinMode(RELAY1_PIN, OUTPUT);
  pinMode(RELAY2_PIN, OUTPUT);
  digitalWrite(RELAY1_PIN, LOW);  // Relay␣
↪off
  digitalWrite(RELAY2_PIN, LOW);  // Relay␣
↪off
}

void loop() {
  // Turn relay 1 on for 2 seconds
  digitalWrite(RELAY1_PIN, HIGH);
  delay(2000);
  digitalWrite(RELAY1_PIN, LOW);
  delay(1000);
```

(continues on next page)

(continued from previous page)

```
  // Turn relay 2 on for 2 seconds
  digitalWrite(RELAY2_PIN, HIGH);
  delay(2000);
  digitalWrite(RELAY2_PIN, LOW);
  delay(1000);
}
```

Warning: Relays can switch high voltage/current. Use appropriate safety precautions and never exceed the relay ratings.

3.8 Next Steps

- Explore example code in the `software/examples/` directory
- Check the pinout documentation for specific pin assignments
- Join the UNIT Electronics community for support and projects
- Visit the GitHub repository for updates and contributions

3.9 Troubleshooting

No Power LEDs

- Check USB-C cable and connection
- Verify microcontroller board is properly mounted
- Check external power supply if used

I2C Devices Not Detected

- Verify devices are properly connected to QWIIC connectors
- Check I2C pull-up resistors are present
- Run I2C scanner to detect device addresses
- Verify correct I2C pins for your platform

Upload Failures

- Check correct board selected in Arduino IDE
- Verify correct COM port selected
- Try pressing reset button before upload
- Check USB cable supports data transfer

Display Not Working

- Verify I2C address (usually 0x3C or 0x3D)
- Check display is properly connected
- Run I2C scanner to confirm device detection
- Verify correct library installed

CHAPTER
FOUR

PLATFORM COMPATIBILITY

4.1 Overview

The DevLab Multi Hub Shield is designed with universal compatibility in mind, supporting multiple popular microcontroller development board formats. This flexibility allows developers to use their preferred platform while accessing the shield's comprehensive feature set.

4.2 Supported Platforms

4.2.1 Pulsar (UNIT Electronics)

Description

The Pulsar is UNIT Electronics' flagship development board featuring:

- ESP32-S3 dual-core processor
- Built-in Wi-Fi and Bluetooth
- USB-C connectivity
- Multiple GPIO pins

Pin Compatibility

The shield provides dedicated pin headers compatible with the Pulsar's pinout, ensuring full access to:

- I2C bus (SDA, SCL)
- SPI bus (MOSI, MISO, SCK, CS)
- UART (TX, RX)
- Analog inputs
- Digital GPIO pins

Example Applications

- IoT sensor hubs with cloud connectivity
- Wireless automation controllers
- Data logging with remote access
- Smart home integration

4.2.2 Seeed Xiao Series

Description

The Xiao series represents ultra-compact development boards with various microcontroller options:

- **Xiao ESP32-C3**: Wi-Fi and Bluetooth LE
- **Xiao ESP32-S3**: Dual-core with AI acceleration
- **Xiao RP2040**: Dual-core ARM Cortex-M0+
- **Xiao SAMD21**: ARM Cortex-M0+ for low power

Pin Compatibility

All Xiao variants share the same pinout, making the shield compatible with:

- 14-pin header configuration
- 3.3V logic level
- I2C, SPI, UART interfaces
- Multiple ADC channels

Power Considerations

The shield provides voltage regulation compatible with Xiao's power requirements. When using battery-powered Xiao boards, the shield can operate from USB-C or external power.

Example Applications

- Wearable sensor platforms
- Battery-powered environmental monitors
- Compact automation controllers
- Educational electronics projects

4.2.3 Raspberry Pi Pico

Description

The Raspberry Pi Pico features the RP2040 microcontroller:

- Dual-core ARM Cortex-M0+ @ 133MHz
- 264KB SRAM
- Flexible GPIO with PIO state machines
- MicroPython and C/C++ support

Pin Compatibility

The shield's Pico-compatible socket provides access to:

- 40-pin dual-row header
- 3.3V and 5V power rails
- Ground connections
- All GPIO pins including SPI, I2C, UART, PWM, ADC

Unique Features

The RP2040's PIO (Programmable I/O) feature can be used with the shield to:

- Implement custom protocols for the LED matrix
- Create precise timing for relay control
- Generate custom waveforms for sensors

Example Applications

- Real-time data acquisition systems
- LED matrix displays and animations
- Multi-sensor environmental monitoring
- Educational robotics platforms

4.2.4 Adafruit Feather

Description

The Feather format is a popular ecosystem with many variants:

- **Feather ESP32**: Wi-Fi and Bluetooth
- **Feather M4 Express**: ARM Cortex-M4 with FPU
- **Feather nRF52840**: Bluetooth LE
- **Feather RP2040**: Dual-core RP2040

Pin Compatibility

The shield supports the standard Feather pinout:

- 28-pin configuration
- LiPo battery charging support (varies by board)
- 3.3V logic level
- Standard I2C, SPI, UART pins

Battery Operation

Many Feather boards include built-in LiPo battery charging. The shield is designed to work with battery-powered Feather boards, making it ideal for portable applications.

Example Applications

- Portable weather stations
- Mobile data loggers
- Battery-powered automation
- Remote sensing applications

4.2.5 ESP32 Devkit V4

Description

The ESP32 Devkit V4 is a popular development board featuring:

- ESP32 dual-core processor
- Wi-Fi 802.11 b/g/n
- Bluetooth Classic and BLE
- Rich peripheral set

Pin Compatibility

The shield provides a socket compatible with the Devkit V4's 38-pin configuration:

- 5V and 3.3V power
- All GPIO pins
- Touch sensor pins
- ADC/DAC channels

Networking Features

The ESP32's networking capabilities combined with the shield enable:

- IoT sensor networks
- Web-based control interfaces
- MQTT data publishing
- Remote monitoring systems

Example Applications

- Smart home hubs

- Industrial IoT gateways
- Environmental monitoring stations
- Wireless sensor networks

4.2.6 Mikro Bus

Description

The Mikro Bus standard by MikroElektronika provides:

- Standardized pinout for Click boards
- Flexible module expansion
- Extensive ecosystem of add-on boards

Pin Compatibility

The shield includes Mikro Bus compatible sockets providing:

- SPI interface
- I2C interface
- UART interface
- PWM, Analog, Interrupt, and Reset pins
- 3.3V and 5V power options

Ecosystem Integration

With Mikro Bus compatibility, users can add:

- Communication modules (LoRa, cellular, etc.)
- Additional sensors
- Display modules
- Motor control boards

Example Applications

- Modular sensor systems
- Expandable control platforms
- Rapid prototyping environments
- Educational modular electronics

4.3 Pin Mapping and Configuration

4.3.1 General Pin Assignments

While specific pin numbers vary by platform, the shield maintains consistent functional assignments:

Table 4.1: Standard Pin Functions

Function	Description
I2C SDA	Primary I2C data line for OLED, QWIIC connectors
I2C SCL	Primary I2C clock line
SPI MOSI	SPI Master Out Slave In
SPI MISO	SPI Master In Slave Out
SPI SCK	SPI Clock
SPI CS	SPI Chip Select (multiple available)
UART TX	Serial transmit
UART RX	Serial receive
ADC	Analog input from potentiometer
PWM	PWM outputs for buzzer and modules
GPIO	Digital I/O for buttons, LEDs, relays

4.3.2 Voltage Level Considerations

3.3V Platforms

- Xiao series
- Raspberry Pi Pico
- Feather boards (most)

5V Tolerant

- Some ESP32 boards
- Platform-specific checking required

Level Shifting

The shield includes appropriate level shifting where necessary to protect 3.3V platforms while interfacing with 5V peripherals.

4.4 Software Support

4.4.1 Arduino IDE

All supported platforms can be programmed using Arduino IDE with appropriate board packages:

- ESP32 boards package
- RP2040 boards package
- SAMD boards package
- Third-party board support

4.4.2 MicroPython

MicroPython support available for:

- Raspberry Pi Pico
- ESP32 variants
- SAMD21/51 (limited)

4.4.3 CircuitPython

Adafruit's CircuitPython supports:

- Feather boards
- Xiao SAMD21
- RP2040-based boards

4.5 Platform Selection Guide

Choose your platform based on:

For Wi-Fi/Bluetooth Projects

- ESP32 Devkit V4 (best range and stability)
- Xiao ESP32-S3 (compact, AI features)
- Feather ESP32 (battery-friendly)

For Low Power Applications

- Xiao SAMD21
- Feather M0
- Pico with sleep modes

For High Performance

- ESP32-S3 (dual-core + AI)
- Feather M4 Express (FPU)
- Pico (PIO state machines)

For Beginners

- Raspberry Pi Pico (excellent documentation)
- Arduino-compatible boards
- Xiao series (simple, affordable)

For Professional/Industrial

- ESP32 Devkit (robust, well-supported)
- Feather (reliable, ecosystem)
- Mikro Bus (modular, expandable)

CHAPTER
FIVE

CONNECTIVITY INTERFACES

5.1 Overview

The DevLab Multi Hub Shield integrates multiple industry-standard communication protocols, providing comprehensive connectivity options for sensors, displays, and peripheral modules. This section details each interface and provides practical guidance for implementation.

5.2 I2C Interface

Description

Inter-Integrated Circuit (I2C) is a synchronous, multi-master, multi-slave serial communication protocol. The shield implements I2C as the primary interface for:

- OLED display communication
- QWIIC/STEMMA QT connectors
- Sensor modules
- Expansion devices

Technical Specifications

Table 5.1: I2C Specifications

Parameter	Value
Protocol	I2C (Two-Wire Interface)
Speed	Standard (100 kHz), Fast (400 kHz)
Voltage Level	3.3V (compatible with 5V via level shifting)
Pull-up Resistors	Integrated on-board (4.7k ohms typical)
Maximum Devices	127 addressable devices
Connectors	7x QWIIC/STEMMA QT (JST 1mm)

Pin Assignment

- **SDA** (Serial Data): Bidirectional data line
- **SCL** (Serial Clock): Clock signal generated by master
- **VCC**: Power supply (3.3V or 5V)
- **GND**: Common ground

Common I2C Addresses

Table 5.2: Typical I2C Device Addresses

Device	Default Address	Alternative
SSD1306 OLED	0x3C	0x3D
BME280 Sensor	0x76	0x77
MPU6050 IMU	0x68	0x69
ADS1115 ADC	0x48	0x49-0x4B
PCA9685 PWM	0x40	0x41-0x7F

Programming Example

```
#include <Wire.h>

void setup() {
  Wire.begin();        // Initialize I2C␣
↪as master
  Serial.begin(115200);
}

void loop() {
  // Read from I2C device at address 0x3C
  Wire.beginTransmission(0x3C);
  Wire.write(0x00);    // Register address
  Wire.endTransmission();

  Wire.requestFrom(0x3C, 1);  // Request 1␣
↪byte
  if (Wire.available()) {
    byte data = Wire.read();
    Serial.println(data, HEX);
  }
  delay(1000);
}
```

5.3 SPI Interface

Description

Serial Peripheral Interface (SPI) is a high-speed, full-duplex synchronous serial communication protocol. The shield provides SPI connectivity for:

- High-speed sensors
- SD card modules
- External memory
- Display modules

Technical Specifications

Table 5.3: SPI Specifications

Parameter	Value
Protocol	SPI (4-wire)
Speed	Up to 20 MHz (platform dependent)
Mode	Master only
Voltage Level	3.3V logic
Chip Selects	Multiple CS pins available

Pin Assignment

- **MOSI** (Master Out Slave In): Data from master to slave
- **MISO** (Master In Slave Out): Data from slave to master
- **SCK** (Serial Clock): Clock signal from master
- **CS/SS** (Chip Select): Device selection (active low)

SPI Modes

Table 5.4: SPI Clock Modes

Mode	CPOL/CPHA	Description
0	0/0	Clock idle low, sample on rising edge
1	0/1	Clock idle low, sample on falling edge
2	1/0	Clock idle high, sample on falling edge
3	1/1	Clock idle high, sample on rising edge

Programming Example

```
#include <SPI.h>

#define CS_PIN 10

void setup() {
  pinMode(CS_PIN, OUTPUT);
  digitalWrite(CS_PIN, HIGH);  // Deselect

  SPI.begin();
  SPI.setClockDivider(SPI_CLOCK_DIV4);  //␣
↪4 MHz
  SPI.setDataMode(SPI_MODE0);
}

void loop() {
  digitalWrite(CS_PIN, LOW);   // Select␣
↪device

  byte response = SPI.transfer(0xAA);  //␣
↪Send and receive

  digitalWrite(CS_PIN, HIGH);  // Deselect
  delay(100);
}
```

5.4 UART Interface

Description

Universal Asynchronous Receiver-Transmitter (UART) provides serial communication for:

- GPS modules
- Bluetooth modules
- Debug console
- Serial sensors

Technical Specifications

Table 5.5: UART Specifications

Parameter	Value
Protocol	Asynchronous serial
Baud Rates	9600, 19200, 38400, 57600, 115200 bps
Data Bits	8 bits (standard)
Parity	None, Even, Odd
Stop Bits	1 or 2
Voltage Level	3.3V TTL

Pin Assignment

- **TX** (Transmit): Data output from shield
- **RX** (Receive): Data input to shield
- **GND**: Common ground reference

Programming Example

```
void setup() {
  Serial.begin(115200);       // USB serial
  Serial1.begin(9600);        // Hardware␣
↪UART
}

void loop() {
  // Forward data between Serial and␣
↪Serial1
  if (Serial.available()) {
    Serial1.write(Serial.read());
  }

  if (Serial1.available()) {
    Serial.write(Serial1.read());
  }
}
```

5.5 GPIO (General Purpose Input/Output)

Description

GPIO pins provide flexible digital input/output functionality for:

- Button reading
- LED control
- Relay activation
- Digital sensors
- Interrupt handling

Technical Specifications

Table 5.6: GPIO Specifications

Parameter	Value
Voltage Level	3.3V logic (5V tolerant on some platforms)
Current per Pin	12-20 mA (platform dependent)
Input Modes	Pull-up, Pull-down, High-impedance
Output Modes	Push-pull, Open-drain
Interrupt Support	Rising edge, Falling edge, Change, Low level

Programming Example

```
#define LED_PIN 13
#define BUTTON_PIN 5

void setup() {
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);

  // Attach interrupt
 ␣
↪attachInterrupt(digitalPinToInterrupt(BUTTON_
↪PIN),
                  buttonPressed, FALLING);
}

void loop() {
  // Main code here
}

void buttonPressed() {
  digitalWrite(LED_PIN, !digitalRead(LED_
↪PIN));
}
```

5.6 ADC (Analog to Digital Converter)

Description

The ADC interface allows reading analog voltages from:

- Potentiometer
- Analog sensors
- Voltage dividers
- Variable resistors

Technical Specifications

Table 5.7: ADC Specifications

Parameter	Value
Resolution	10-12 bits (platform dependent)
Voltage Range	0 - 3.3V (typical)
Channels	Multiple ADC inputs available
Sampling Rate	Up to 200 ksps (platform dependent)
Reference Voltage	Internal or external

Programming Example

```
#define POT_PIN A0

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);  // 12-bit␣
↪resolution (if supported)
}

void loop() {
  int rawValue = analogRead(POT_PIN);
  float voltage = rawValue * (3.3 / 4095.
↪0);

  Serial.print("Raw: ");
  Serial.print(rawValue);
  Serial.print(" Voltage: ");
  Serial.println(voltage);

  delay(100);
}
```

5.7 Interface Selection Guide

When to Use I2C

- Multiple devices on same bus
- Low to moderate data rates
- Sensor modules
- Display communication
- QWIIC/STEMMA QT ecosystem

When to Use SPI

- High-speed data transfer
- SD cards, memory chips
- Fast displays
- Real-time applications
- Point-to-point communication

When to Use UART

- GPS modules
- Wireless modules (Bluetooth, Wi-Fi)
- Debug output
- Simple two-wire communication
- Legacy serial devices

When to Use GPIO

- Simple on/off control
- Button inputs
- LED outputs
- Interrupt-based events
- Bit-banging custom protocols

5.8 Best Practices

I2C

- Always check for address conflicts
- Use I2C scanner to detect devices
- Keep cable lengths short (<30cm for reliable operation)
- Verify pull-up resistors are present

SPI

- Use shortest possible wires for high-speed operation
- Properly manage chip select signals
- Check SPI mode compatibility
- Consider signal integrity at high speeds

UART

- Match baud rates exactly between devices
- Use common ground reference
- Consider flow control for reliable communication
- Avoid ground loops in long cable runs

GPIO

- Never exceed maximum current ratings

- Use current-limiting resistors for LEDs
- Implement proper debouncing for buttons
- Be cautious with voltage levels (3.3V vs 5V)

ADC

- Filter noisy signals in hardware
- Average multiple readings for stability
- Consider reference voltage accuracy
- Account for input impedance

CHAPTER
SIX

LED MATRIX

6.1 Overview

The DevLab Multi Hub Shield features a 5x5 RGB LED matrix that provides visual feedback, simple graphics display, and animation capabilities. Each LED is individually addressable and can display any color in the RGB spectrum.

6.2 Technical Specifications

Table 6.1: LED Matrix Specifications

Parameter	Value
Configuration	5x5 matrix (25 LEDs total)
LED Type	RGB (Red, Green, Blue)
Control	Individually addressable
Colors	16.7 million colors (24-bit)
Brightness	Adjustable per LED
Protocol	GPIO-controlled or addressable (WS2812/SK6812 compatible)
Voltage	5V supply for LEDs
Current	~60mA per LED at full white

6.3 Features

Individual Control

Each of the 25 LEDs can be: - Set to any RGB color independently - Adjusted for brightness - Turned on or off individually - Updated at high refresh rates

Color Capabilities

- Full RGB spectrum (16.7M colors)
- Adjustable color temperature
- Smooth color transitions
- Color mixing and blending

Animation Support

- Scrolling text
- Pattern displays
- Animations and effects
- Game displays
- Status indicators

6.4 Pin Configuration

The LED matrix connects to the microcontroller via:

- **Data Pin**: Single data line for addressable LEDs
- **Power (5V)**: LED power supply
- **Ground**: Common ground

6.5 Programming Examples

6.5.1 Using FastLED Library (Arduino)

```
#include <FastLED.h>

#define LED_PIN     6
#define NUM_LEDS    25
#define BRIGHTNESS  50
#define LED_TYPE    WS2812B
#define COLOR_ORDER GRB

CRGB leds[NUM_LEDS];

void setup() {
  FastLED.addLeds<LED_TYPE, LED_PIN, COLOR_
↪ORDER>(leds, NUM_LEDS);
  FastLED.setBrightness(BRIGHTNESS);
}

void loop() {
  // Set all LEDs to red
  fill_solid(leds, NUM_LEDS, CRGB::Red);
```

(continues on next page)

(continued from previous page)

```
  FastLED.show();
  delay(1000);

  // Set all LEDs to green
  fill_solid(leds, NUM_LEDS, CRGB::Green);
  FastLED.show();
  delay(1000);

  // Set all LEDs to blue
  fill_solid(leds, NUM_LEDS, CRGB::Blue);
  FastLED.show();
  delay(1000);
}
```

6.5.2 Individual LED Control

```
void setup() {
  FastLED.addLeds<WS2812B, LED_PIN, GRB>
↪(leds, NUM_LEDS);
  FastLED.setBrightness(50);
}

void loop() {
  // Set specific LEDs
  leds[0] = CRGB::Red;        // Top-left␣
↪corner
  leds[4] = CRGB::Blue;       // Top-right␣
↪corner
  leds[20] = CRGB::Green;     // Bottom-
↪left corner
  leds[24] = CRGB::Yellow;    // Bottom-
↪right corner
  leds[12] = CRGB::White;     // Center LED

  FastLED.show();
  delay(2000);

  // Clear all
  FastLED.clear();
  FastLED.show();
  delay(1000);
}
```

6.5.3 LED Matrix Mapping

5x5 matrix layout (LED indices):

```
[0]  [1]  [2]  [3]  [4]
[5]  [6]  [7]  [8]  [9]
[10] [11] [12] [13] [14]
[15] [16] [17] [18] [19]
[20] [21] [22] [23] [24]
```

Helper function to access by row/column:

```
int getLEDIndex(int row, int col) {
  return row * 5 + col;
}

void setPixel(int row, int col, CRGB␣
↪color) {
  if (row >= 0 && row < 5 && col >= 0 &&␣
↪col < 5) {
    leds[getLEDIndex(row, col)] = color;
  }
}
```

6.6 Animation Examples

6.6.1 Rainbow Effect

```
void rainbow() {
  static uint8_t hue = 0;

  for(int i = 0; i < NUM_LEDS; i++) {
    leds[i] = CHSV(hue + (i * 10), 255,␣
↪255);
  }

  FastLED.show();
  hue++;
  delay(20);
}
```

6.6.2 Breathing Effect

```
void breathe(CRGB color) {
  static int brightness = 0;
  static int direction = 1;

  fill_solid(leds, NUM_LEDS, color);
  FastLED.setBrightness(brightness);
  FastLED.show();

  brightness += direction * 5;
  if (brightness >= 255 || brightness <=␣
↪0) {
    direction *= -1;
  }

  delay(30);
}
```

6.6.3 Scrolling Pattern

```
void scrollPattern() {
  static int offset = 0;

  FastLED.clear();

  for(int i = 0; i < 5; i++) {
    int pos = (i + offset) % 5;
    leds[getLEDIndex(pos, i)] = CRGB::Cyan;
  }

  FastLED.show();
  offset = (offset + 1) % 5;
  delay(200);
}
```

6.6.4 Display Patterns

```
// Smiley face
void showSmiley() {
  FastLED.clear();

  // Eyes
  leds[getLEDIndex(1, 1)] = CRGB::Yellow;
  leds[getLEDIndex(1, 3)] = CRGB::Yellow;

  // Smile
  leds[getLEDIndex(3, 1)] = CRGB::Yellow;
  leds[getLEDIndex(3, 3)] = CRGB::Yellow;
  leds[getLEDIndex(4, 2)] = CRGB::Yellow;
```

(continues on next page)

(continued from previous page)

```
  FastLED.show();
}

// Cross pattern
void showCross() {
  FastLED.clear();

  for(int i = 0; i < 5; i++) {
    leds[getLEDIndex(i, 2)] = CRGB::Red;  /
↪/ Vertical
    leds[getLEDIndex(2, i)] = CRGB::Red;  /
↪/ Horizontal
  }

  FastLED.show();
}
```

6.6.5 MicroPython Example

```
from machine import Pin
import neopixel
import time

# Initialize LED matrix
np = neopixel.NeoPixel(Pin(6), 25)

# Set color (R, G, B)
def set_pixel(index, r, g, b):
    np[index] = (r, g, b)
    np.write()

# Rainbow cycle
def rainbow():
    for i in range(25):
        hue = int((i / 25) * 255)
        np[i] = hue_to_rgb(hue)
    np.write()

# Clear all LEDs
def clear():
    for i in range(25):
        np[i] = (0, 0, 0)
    np.write()
```

6.7 Practical Applications

6.7.1 Status Indicators

- System status visualization
- Progress bars
- Battery level indicators
- Connection status
- Error/warning displays

6.7.2 Games and Entertainment

- Simple games (Snake, Pong)
- Reaction time games
- Pattern matching
- Music visualizer
- Dice simulator

6.7.3 Educational Projects

- Learning RGB color mixing
- Animation programming
- Pattern recognition
- Visual feedback for sensors
- Binary number display

6.7.4 Data Visualization

- Bar graphs
- Temperature indicators
- Sound level meters
- Sensor data representation
- Real-time monitoring

6.8 Best Practices

Power Considerations

- Limit brightness when running from USB (500mA limit)
- Use external power for full brightness
- Calculate total current: LEDs × 60mA × brightness factor

Performance

- Update LEDs only when needed
- Use FastLED.show() sparingly
- Batch updates for smooth animation
- Optimize color calculations

Color Selection

- Use HSV for easier color manipulation
- Pre-calculate color palettes
- Consider color blindness accessibility
- Test colors in actual lighting conditions

Programming Tips

- Create reusable animation functions
- Use arrays for pattern storage
- Implement state machines for complex displays
- Add user controls for brightness/speed

6.9 Power Consumption

Table 6.2: Current Consumption Examples

Scenario	Current Draw	Notes
All LEDs off	~1 mA	Standby current
5 LEDs at 50% white	~150 mA	Typical usage
25 LEDs at 100% white	~1500 mA	Maximum draw
25 LEDs at 25% brightness	~375 mA	Recommended for USB power

6.10 Troubleshooting

LEDs not lighting

- Check data pin connection
- Verify 5V power supply
- Confirm correct LED_PIN definition
- Test with simple example first

Wrong colors displayed

- Check COLOR_ORDER setting (RGB vs GRB)
- Verify data line integrity
- Test individual color channels
- Check for power supply issues

Flickering or random colors

- Add capacitor (100-1000µF) across power
- Shorten data wire if possible
- Add 330 ohm resistor in data line
- Ensure stable power supply

First LED works, others don't

- Check for broken LED in chain
- Verify data passes through each LED
- Test continuity of data line
- Replace faulty LED if needed

CHAPTER
SEVEN

DISPLAY HUB - OLED DISPLAY

7.1 Overview

The DevLab Multi Hub Shield features an integrated 128x64 I2C OLED display that provides a compact, high-contrast visual interface for displaying text, graphics, and sensor data. The display uses the SSD1306 controller and communicates via the I2C bus.

7.2 Technical Specifications

Table 7.1: OLED Display Specifications

Parameter	Value
Resolution	128 x 64 pixels
Controller	SSD1306 (or compatible)
Interface	I2C
I2C Address	0x3C (default) or 0x3D
Display Type	OLED (Organic LED)
Color	Monochrome (White/Blue/Yellow)
Viewing Angle	>160 degrees
Voltage	3.3V logic, 7-9V internal charge pump
Current	~20mA typical
Brightness	Adjustable (0-255)
Power	Low power consumption

7.3 Display Features

High Contrast

- OLED technology provides true black pixels
- No backlight needed
- Excellent readability in various lighting conditions
- High contrast ratio (>2000:1)

Fast Response

- No ghosting or motion blur
- Instant pixel switching
- Suitable for animations
- Smooth scrolling capabilities

Wide Viewing Angle

- Readable from extreme angles
- No color shift
- Consistent brightness across viewing angles

7.4 Pin Connections

The OLED display connects via I2C:

- **SDA**: I2C Data line
- **SCL**: I2C Clock line
- **VCC**: 3.3V Power
- **GND**: Ground

These are shared with the QWIIC/STEMMA connectors on the same I2C bus.

7.5 Programming Libraries

7.5.1 Arduino Libraries

Adafruit SSD1306 (Recommended)

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1  // No reset pin
#define SCREEN_ADDRESS 0x3C

Adafruit_SSD1306 display(SCREEN_WIDTH,␣
↪SCREEN_HEIGHT, &Wire, OLED_RESET);
```

U8g2 Library (Alternative)

```
#include <U8g2lib.h>

U8G2_SSD1306_128X64_NONAME_F_HW_I2C␣
↪u8g2(U8G2_R0, U8X8_PIN_NONE);
```

7.6 Basic Examples

7.6.1 Initialization

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1

Adafruit_SSD1306 display(SCREEN_WIDTH,␣
↪SCREEN_HEIGHT, &Wire, OLED_RESET);

void setup() {
  Serial.begin(115200);

  // Initialize display
  if(!display.begin(SSD1306_SWITCHCAPVCC,␣
↪0x3C)) {
    Serial.println(F("SSD1306 allocation␣
↪failed"));
    for(;;);  // Don't proceed, loop forever
  }

  // Clear the buffer
  display.clearDisplay();
  display.display();
}
```

7.6.2 Text Display

```
void displayText() {
  display.clearDisplay();

  // Set text properties
  display.setTextSize(1);              //␣
↪Normal 1:1 pixel scale
  display.setTextColor(SSD1306_WHITE); //␣
↪White text
  display.setCursor(0, 0);             //␣
```

(continues on next page)

(continued from previous page)

```
↪Start at top-left

  // Display text
  display.println(F("DevLab Shield"));
  display.println(F("Multi Hub"));
  display.println();
  display.setTextSize(2);
  display.println(F("Hello!"));

  // Update display
  display.display();
}
```

7.6.3 Multiple Text Sizes

```
void multiSizeText() {
  display.clearDisplay();

  // Size 1 (small)
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("Size 1: Small");

  // Size 2 (medium)
  display.setTextSize(2);
  display.setCursor(0, 16);
  display.println("Size 2");

  // Size 3 (large)
  display.setTextSize(3);
  display.setCursor(0, 40);
  display.println("Big");

  display.display();
}
```

7.7 Graphics Examples

7.7.1 Drawing Shapes

```
void drawShapes() {
  display.clearDisplay();

  // Draw rectangle
  display.drawRect(0, 0, 60, 30, SSD1306_
↪WHITE);

  // Draw filled rectangle
```

(continues on next page)

(continued from previous page)

```
  display.fillRect(70, 0, 58, 30, SSD1306_
↪WHITE);

  // Draw circle
  display.drawCircle(30, 48, 15, SSD1306_
↪WHITE);

  // Draw filled circle
  display.fillCircle(98, 48, 15, SSD1306_
↪WHITE);

  display.display();
}
```

7.7.2 Drawing Lines

```
void drawLines() {
  display.clearDisplay();

  for(int16_t i=0; i<display.width();␣
↪i+=4) {
    display.drawLine(0, 0, i, display.
↪height()-1, SSD1306_WHITE);
  }

  for(int16_t i=0; i<display.height();␣
↪i+=4) {
    display.drawLine(0, 0, display.width()-
↪1, i, SSD1306_WHITE);
  }

  display.display();
}
```

7.7.3 Displaying Bitmaps

```
// Define bitmap (16x16 pixels)
const unsigned char icon_16x16 [] PROGMEM␣
↪= {
  0x00, 0x00, 0x07, 0xe0, 0x18, 0x18, 0x20,
↪ 0x04,
  0x47, 0xe2, 0x48, 0x12, 0x90, 0x09, 0x90,
↪ 0x09,
  0x90, 0x09, 0x90, 0x09, 0x48, 0x12, 0x47,
↪ 0xe2,
  0x20, 0x04, 0x18, 0x18, 0x07, 0xe0, 0x00,
↪ 0x00
};
```

(continues on next page)

(continued from previous page)

```
void displayIcon() {
  display.clearDisplay();
  display.drawBitmap(56, 24, icon_16x16,␣
↪16, 16, SSD1306_WHITE);
  display.display();
}
```

7.8 Sensor Data Display

7.8.1 Temperature Display

```
void displayTemperature(float temp) {
  display.clearDisplay();

  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("Temperature:");

  display.setTextSize(3);
  display.setCursor(10, 20);
  display.print(temp, 1);

  display.setTextSize(2);
  display.println(" C");

  display.display();
}
```

7.8.2 Multi-Sensor Dashboard

```
void displayDashboard(float temp, float␣
↪hum, float press) {
  display.clearDisplay();

  display.setTextSize(1);

  // Temperature
  display.setCursor(0, 0);
  display.print("Temp: ");
  display.print(temp, 1);
  display.println(" C");

  // Humidity
  display.setCursor(0, 12);
  display.print("Hum:  ");
  display.print(hum, 1);
  display.println(" %");
```

(continues on next page)

(continued from previous page)

```
  // Pressure
  display.setCursor(0, 24);
  display.print("Press:");
  display.print(press, 0);
  display.println(" hPa");

  // Time or status
  display.setCursor(0, 40);
  display.println("Status: OK");

  display.display();
}
```

7.8.3 Progress Bar

```
void displayProgress(int percent) {
  display.clearDisplay();

  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("Progress:");

  // Draw progress bar outline
  display.drawRect(10, 20, 108, 12,␣
↪SSD1306_WHITE);

  // Fill progress
  int fillWidth = map(percent, 0, 100, 0,␣
↪104);
  display.fillRect(12, 22, fillWidth, 8,␣
↪SSD1306_WHITE);

  // Display percentage
  display.setCursor(50, 40);
  display.setTextSize(2);
  display.print(percent);
  display.print("%");

  display.display();
}
```

7.9 Advanced Features

7.9.1 Scrolling Text

```
void setupScrolling() {
  display.clearDisplay();
  display.setTextSize(2);
  display.setCursor(0, 0);
  display.println("Scrolling");
  display.println("Text!");
  display.display();

  // Scroll left
  display.startscrollleft(0x00, 0x0F);
  delay(2000);
  display.stopscroll();

  // Scroll right
  display.startscrollright(0x00, 0x0F);
  delay(2000);
  display.stopscroll();
}
```

7.9.2 Inverted Display

```
void invertDisplay() {
  display.invertDisplay(true);  // Invert
  delay(1000);
  display.invertDisplay(false); // Normal
}
```

7.9.3 Brightness Control

```
void setBrightness(uint8_t brightness) {
  // brightness: 0-255
  display.ssd1306_command(SSD1306_
↪SETCONTRAST);
  display.ssd1306_command(brightness);
}
```

7.10 MicroPython Example

```
from machine import Pin, I2C
import ssd1306
import time

# Initialize I2C
i2c = I2C(0, scl=Pin(22), sda=Pin(21),␣
↪freq=400000)

# Initialize display
oled = ssd1306.SSD1306_I2C(128, 64, i2c,␣
↪addr=0x3C)

# Clear display
oled.fill(0)
oled.show()

# Display text
oled.text('DevLab Shield', 0, 0)
oled.text('Multi Hub', 0, 10)
oled.text('Hello World!', 0, 30)
oled.show()

# Simple animation
for i in range(0, 128, 4):
    oled.fill(0)
    oled.rect(i, 28, 16, 16, 1)
    oled.show()
    time.sleep(0.05)
```

7.11 TFT Display Configuration

The shield supports TFT display customization for enhanced color graphics capabilities.

TFT Display Options

- Color TFT displays via SPI interface
- Touch screen support
- Higher resolution options
- Additional header connections required

Configuration

Consult the shield documentation for: - Specific TFT models supported - Jumper/switch settings - Pin assignments for SPI TFT - Touch controller integration

7.12 Best Practices

Performance

- Update display only when needed (not every loop)
- Use partial updates when possible
- Clear only the changed area
- Batch drawing operations before display.display()

Power Management

- Turn off display when not needed
- Reduce brightness for battery operation
- Use sleep modes in low-power applications

Display Care

- Avoid displaying static images for extended periods
- Vary content to prevent burn-in
- Use screensavers for always-on applications

Memory

- Use F() macro for string literals to save RAM
- Consider display buffer size (1KB for 128x64)
- Use PROGMEM for bitmaps and fonts

7.13 Troubleshooting

Display Not Working

- Check I2C address (0x3C vs 0x3D)
- Verify I2C connections (SDA, SCL)
- Run I2C scanner
- Check power supply
- Verify library installation

Garbled Display

- Wrong I2C address
- I2C bus speed too high
- Power supply issues
- Bad connections

Dim Display

- Adjust brightness/contrast
- Check power supply voltage
- Verify VCC is 3.3V, not 5V

- Display may be failing

Display Freezes

- Check for infinite loops
- Verify I2C bus isn't stuck
- Reset microcontroller
- Check for memory issues

7.14 Applications

- Sensor data visualization
- System status display
- Menu systems
- Data logging display
- Animations and graphics
- User interface
- Debugging output
- Real-time monitoring

CHAPTER
EIGHT

DEVLAB ECOSYSTEM INTEGRATION

8.1 devlab concept

discover unit electronics' devlab ecosystem, its universal connector, and how it facilitates rapid and modular prototyping.

8.1.1 introduction

devlab is a modular ecosystem proposed by unit electronics, geared towards rapid prototyping, immediate assembly and flexibility in electronic design.

its purpose is to accelerate hardware development through integrated modules, sensors, and tools that share a common interconnection standard, without imposing restrictions on the physical form factor of the boards. the key element of the devlab ecosystem is the universal 1.0 mm, 4-pin jst sh connector, combined with a development part number nomenclature, which formally identifies products belonging to the ecosystem.

8.1.2 definition

devlab is a development ecosystem composed of electronic modules, sensors, boards, and accessories that meet these essential principles:

Table 8.1: devlab core principles

feature	definition
rapid prototyping	immediate integration with minimal configuration
simple assembly	direct connection between modules using a standard connector
pcb free format	there is no mandatory size or shape

8.2 devlab connector

the devlab universal connector is based on a 4-pin jst sh 1.0mm and is used as a common interface for communication, control, and power.

Note: the devlab connector is multiprotocol, but not simultaneous; this means that only one protocol should be used per connector at a time.

8.2.1 compatible protocols

the connector allows operation with various protocols depending on the module:

- **i2c** (sda/scl)
- **uart** (rx/tx)
- **pwm**
- **analog inputs** (adc)
- **multipurpose gpio**
- **rgb led control**
- **swd/io** for programming and debugging

8.2.2 purpose

the devlab connector functions as a multiprotocol interface, allowing:

- connect sensors
- power modules
- pwm outputs
- converters
- programmable devices
- expanders or multiplexers
- purification systems

> **Warning:** **technical note**: the devlab connector is multiprotocol, but not simultaneous. only one protocol should be used per connector at a time; many of the signals are not compatible with each other when mixed in the same module or cable. the function of the connector will depend on the design of each module and the pin assignment made on the main board.

8.2.3 official recommendation

to avoid electrical or communication conflicts, it is recommended to use a single protocol for each devlab connector, especially when the connected modules come from different families (i2c, uart, pwm, swd, etc.).

this rule ensures the integrity of the ecosystem, protects devices, and guarantees predictable and safe operation.

8.3 design philosophy

devlab does not define a single form factor like arduino r3, feather, or raspberry pi pico. devlab modules can have any mechanical design: round, rectangular, miniature, large-scale, and intended for assembly into end products.

the only mandatory standard is to include the devlab connector.

8.3.1 exceptions

there are boards that use the devlab connector, but they do not officially belong to the ecosystem because their development part number does not include the devlab nomenclature.

Table 8.2: compatible non-devlab boards

development part number	product
ue0071	pulsar h2 development board
ue0072	touch dot s3
ue0081	jun r3 development board

8.3.2 first generation

the first implementation of the devlab concept occurred with the dualmcu family, considered the first generation, where the following was introduced:

- the use of the devlab universal connector
- the integration of multiple microcontrollers
- the multi-protocol philosophy
- compatibility with existing formats (nano, r3, circular)
- the concept of interconnectable independent modules

8.4 official identification

for a product to be officially devlab-certified, it must:

Table 8.3: certification requirements

number	requirement
1	include the word “devlab” in your official name
2	be registered with a development part number within the ue00xx series
3	integrate the devlab connector
4	maintain the modular, multi-protocol and rapid prototyping philosophy

8.5 qwiic/stemma qt compatibility

the devlab multi hub shield features 7 qwiic-stemma compatible connectors (jst 1mm pitch) that enable seamless integration with the devlab ecosystem and third-party i2c modules. this standardized connector system allows for tool-free, daisy-chainable connections to hundreds of sensors and modules.

8.5.1 what is qwiic/stemma qt?

qwiic (developed by sparkfun) and stemma qt (developed by adafruit) are compatible i2c connector standards that share:

- same jst sh 4-pin connector (1mm pitch)
- identical pinout
- cross-compatible modules
- standardized i2c communication

8.5.2 benefits

- no soldering required
- hot-pluggable connections
- daisy-chain multiple devices
- standardized voltage (3.3v)
- polarity protection
- wide ecosystem compatibility

8.6 technical specifications

Table 8.4: qwiic/stemma specifications

parameter	value
connector type	jst sh 1mm pitch
number of connectors	7 ports
pins	4 (vcc, gnd, sda, scl)
voltage	3.3v
current per port	500ma max (total current shared)
i2c speed	up to 400 khz (fast mode)
cable length	up to 1m recommended
daisy-chain	unlimited (limited by i2c address space)

8.7 pinout

standard 4-pin qwiic/stemma qt pinout:

Table 8.5: connector pinout

pin	signal	function	wire color
1	gnd	ground	black
2	3.3v	power supply	red
3	sda	i2c data	blue
4	scl	i2c clock	yellow

color coding

standard cable colors: - **black**: ground - **red**: 3.3v power - **blue**: i2c data (sda) - **yellow**: i2c clock (scl)

8.8 Compatible Modules

The shield is compatible with hundreds of QWIIC and STEMMA QT modules:

8.8.1 Environmental Sensors

Table 8.6: Popular Environmental Sensors

Module	Measurements	I2C Address
BME280	Temperature, Humidity, Pressure	0x76/0x77
BMP388	Pressure, Temperature	0x76/0x77
SHT31	Temperature, Humidity	0x44/0x45
ENS160	Air Quality (eCO2, TVOC)	0x52/0x53
VEML7700	Ambient Light	0x10

8.8.2 Motion and Orientation

Table 8.7: Motion Sensors

Module	Features	I2C Address
MPU6050	6-axis IMU (Accel + Gyro)	0x68/0x69
BNO085	9-axis IMU with fusion	0x4A/0x4B
LIS3DH	3-axis Accelerometer	0x18/0x19
VL53L1X	Time-of-Flight Distance	0x29

8.8.3 Display Modules

Table 8.8: Display Options

Module	Type	I2C Address
SSD1306	128x64 OLED	0x3C/0x3D
SH1107	128x128 OLED	0x3C/0x3D
HT16K33	LED Matrix Driver	0x70-0x77

8.8.4 Input/Output Expansion

Table 8.9: I/O Modules

Module	Features	I2C Address
PCA9685	16-channel PWM	0x40-0x7F
MCP23017	16-bit GPIO Expander	0x20-0x27
ADS1115	16-bit ADC (4-channel)	0x48-0x4B

8.8.5 Other Useful Modules

- GPS modules (PA1010D)
- RTC (Real-Time Clock) modules
- EEPROM storage modules
- DAC (Digital-to-Analog) modules
- Color sensors (TCS34725)
- Gesture sensors (APDS9960)

8.9 Daisy-Chaining

Concept

Multiple QWIIC modules can be connected in series using the dual connectors on each module:

```
Shield → Module 1 → Module 2 → Module 3␣
↪→ ...

Each module has 2 QWIIC ports for daisy-
↪chaining
```

Configuration Example

```
DevLab Shield
      │
      ├── BME280 (Temp/Humidity/Pressure)
      │     └── SSD1306 (OLED Display)
      │           └── MPU6050 (IMU)
      │
      ├── VL53L1X (Distance Sensor)
      │
      └── ADS1115 (ADC Module)
```

Advantages

- Single I2C bus for all devices
- Minimal wiring
- Easy to add/remove modules
- No breadboard needed
- Clean installations

8.10 Programming Examples

8.10.1 Scanning for Devices

```
#include <Wire.h>

void setup() {
  Wire.begin();
  Serial.begin(115200);
  delay(1000);
  Serial.println("I2C Scanner");
}

void loop() {
  byte error, address;
  int nDevices = 0;

  Serial.println("Scanning...");

  for(address = 1; address < 127;␣
↪address++) {
    Wire.beginTransmission(address);
    error = Wire.endTransmission();

    if (error == 0) {
      Serial.print("Device found at 0x");
      if (address < 16) Serial.print("0");
      Serial.println(address, HEX);
      nDevices++;
    }
  }

  if (nDevices == 0)
    Serial.println("No devices found");
  else
    Serial.println("Scan complete");

  delay(5000);
}
```

8.10.2 Multi-Sensor Example

```
#include <Wire.h>
#include <Adafruit_BME280.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_SSD1306.h>

Adafruit_BME280 bme;
Adafruit_MPU6050 mpu;
Adafruit_SSD1306 display(128, 64, &Wire, -
↪1);

void setup() {
  Wire.begin();
  Serial.begin(115200);

  // Initialize sensors
  if (!bme.begin(0x76)) {
    Serial.println("BME280 not found!");
  }

  if (!mpu.begin()) {
    Serial.println("MPU6050 not found!");
  }

  if (!display.begin(SSD1306_SWITCHCAPVCC,␣
↪0x3C)) {
    Serial.println("Display not found!");
  }

  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
}

void loop() {
  // Read BME280
  float temp = bme.readTemperature();
  float humidity = bme.readHumidity();
  float pressure = bme.readPressure() /␣
↪100.0F;

  // Read MPU6050
  sensors_event_t a, g, temp_event;
  mpu.getEvent(&a, &g, &temp_event);

  // Display on OLED
  display.clearDisplay();
  display.setCursor(0, 0);
  display.print("Temp: ");
  display.print(temp, 1);
  display.println(" C");
  display.print("Hum: ");
```

(continues on next page)

(continued from previous page)

```
  display.print(humidity, 1);
  display.println(" %");
  display.print("AccX: ");
  display.println(a.acceleration.x, 1);
  display.display();

  delay(1000);
}
```

8.10.3 MicroPython Example

```
from machine import Pin, I2C
import bme280
import ssd1306
import time

# Initialize I2C
i2c = I2C(0, scl=Pin(22), sda=Pin(21))

# Scan for devices
devices = i2c.scan()
print("I2C devices found:", [hex(d) for d␣
↪in devices])

# Initialize sensors
bme = bme280.BME280(i2c=i2c, address=0x76)
oled = ssd1306.SSD1306_I2C(128, 64, i2c,␣
↪addr=0x3C)

while True:
    # Read sensor
    temp, pressure, humidity = bme.read_
↪compensated_data()

    # Display
    oled.fill(0)
    oled.text(f"Temp: {temp/100:.1f}C", 0,␣
↪0)
    oled.text(f"Hum: {humidity/1024:.1f}%",
↪ 0, 10)
    oled.show()

    time.sleep(1)
```

8.11 Best Practices

Addressing Conflicts

- Check I2C addresses before connecting multiple modules
- Some modules allow address configuration via jumpers
- Use I2C scanner to verify all devices
- Document your module addresses

Cable Management

- Keep cables as short as practical
- Avoid running I2C cables near power lines
- Use cable clips for organization
- Label cables for easier troubleshooting

Power Considerations

- Calculate total current draw of all modules
- Don't exceed 500mA total on QWIIC bus
- Use external power for high-current modules
- Consider voltage drops on long cables

Reliability

- Secure connectors to prevent disconnection
- Test each module individually first
- Update firmware/libraries regularly
- Use quality QWIIC cables

8.12 Common Issues and Solutions

Device Not Detected

- Verify cable connections
- Check module power LED
- Run I2C scanner
- Try different QWIIC port
- Check I2C pull-up resistors

Communication Errors

- Reduce cable length
- Lower I2C clock speed
- Check for address conflicts
- Verify power supply stability
- Add decoupling capacitors

Intermittent Operation

- Secure loose connections
- Check cable quality
- Verify power supply capacity
- Reduce electromagnetic interference
- Test individual modules

8.13 Project Ideas

8.13.1 Weather Station

Modules needed: - BME280 (Temperature, Humidity, Pressure) - VEML7700 (Light sensor) - SSD1306 (Display) - RTC module (Time stamping)

8.13.2 Motion Tracker

Modules needed: - MPU6050 or BNO085 (IMU) - GPS module - MicroSD module (Data logging) - Display module

8.13.3 Air Quality Monitor

Modules needed: - ENS160 (Air quality sensor) - BME280 (Environmental data) - SSD1306 (Display) - RGB LED module (Visual alerts)

8.13.4 Robotic Sensor Array

Modules needed: - Multiple VL53L1X (Distance sensors) - IMU module (Orientation) - GPIO expander (Control outputs) - PWM driver (Motor control)

8.14 Ecosystem Resources

SparkFun QWIIC - Product catalog: https://www.sparkfun.com/qwiic - Tutorials and guides - Open-source designs

Adafruit STEMMA QT - Product catalog: https://www.adafruit.com/category/620 - Learn guides - CircuitPython libraries

Third-Party Modules - DFRobot Gravity sensors - Seeed Grove I2C modules - Many compatible designs

CHAPTER
NINE

RELAY MODULES

9.1 Overview

The DevLab Multi Hub Shield integrates 2 electromechanical relay modules that enable control of high-voltage and high-current loads. These relays act as electrically isolated switches, allowing low-voltage microcontrollers to safely control AC/DC appliances, motors, lights, and other power devices.

9.2 Technical Specifications

Table 9.1: Relay Specifications

Parameter	Value
Number of Relays	2 independent modules
Type	SPDT (Single Pole Double Throw)
Coil Voltage	5V DC
Contact Rating	10A @ 250VAC, 10A @ 30VDC
Control Signal	3.3V/5V logic compatible
Isolation	Optically isolated control
Switching Time	<10ms
Terminals	Screw terminals
Indicators	LED status per relay

9.3 Relay Construction

SPDT Configuration

Each relay has three terminals:

- **COM** (Common): Input terminal
- **NO** (Normally Open): Closed when relay is activated
- **NC** (Normally Closed): Open when relay is activated

```
Relay OFF (Deenergized):
COM --+-- NC (connected)
      +-- NO (disconnected)

Relay ON (Energized):
COM --+-- NC (disconnected)
      +-- NO (connected)
```

Terminal Layout

```
+-----------------+
|  COM  NC   NO   |  Relay 1
|   o    o    o   |
+-----------------+

+-----------------+
|  COM  NC   NO   |  Relay 2
|   o    o    o   |
+-----------------+
```

9.4 Control Interface

Digital Control

Each relay is controlled by a single GPIO pin:

- **HIGH (1)**: Relay energized (NO closed, NC open)
- **LOW (0)**: Relay de-energized (NO open, NC closed)

Pin Assignment

- **Relay 1 Control Pin**: GPIO (platform specific)
- **Relay 2 Control Pin**: GPIO (platform specific)

Status LEDs

- Each relay has an onboard LED indicator
- LED ON: Relay energized
- LED OFF: Relay de-energized

9.5 Programming Examples

9.5.1 Basic Control (Arduino)

```
// Define relay control pins
#define RELAY1_PIN 7
#define RELAY2_PIN 8

void setup() {
  pinMode(RELAY1_PIN, OUTPUT);
  pinMode(RELAY2_PIN, OUTPUT);

  // Initialize relays to OFF state
  digitalWrite(RELAY1_PIN, LOW);
  digitalWrite(RELAY2_PIN, LOW);
}

void loop() {
  // Turn relay 1 ON for 2 seconds
  digitalWrite(RELAY1_PIN, HIGH);
  delay(2000);
  digitalWrite(RELAY1_PIN, LOW);
  delay(1000);

  // Turn relay 2 ON for 2 seconds
  digitalWrite(RELAY2_PIN, HIGH);
  delay(2000);
  digitalWrite(RELAY2_PIN, LOW);
  delay(1000);
}
```

9.5.2 Timed Control

```
unsigned long relay1Timer = 0;
unsigned long relay2Timer = 0;
bool relay1State = false;
bool relay2State = false;

void setup() {
  pinMode(RELAY1_PIN, OUTPUT);
  pinMode(RELAY2_PIN, OUTPUT);
  digitalWrite(RELAY1_PIN, LOW);
  digitalWrite(RELAY2_PIN, LOW);
}

void loop() {
  unsigned long currentTime = millis();

  // Relay 1: ON for 5 seconds every 10␣
↪seconds
```

(continues on next page)

(continued from previous page)

```
  if (!relay1State && currentTime -␣
↪relay1Timer >= 10000) {
    digitalWrite(RELAY1_PIN, HIGH);
    relay1State = true;
    relay1Timer = currentTime;
  } else if (relay1State && currentTime -␣
↪relay1Timer >= 5000) {
    digitalWrite(RELAY1_PIN, LOW);
    relay1State = false;
    relay1Timer = currentTime;
  }

  // Relay 2: Similar logic with different␣
↪timing
}
```

9.5.3 Button Control

```
#define BUTTON1_PIN 2
#define BUTTON2_PIN 3
#define RELAY1_PIN 7
#define RELAY2_PIN 8

bool relay1State = false;
bool relay2State = false;

void setup() {
  pinMode(BUTTON1_PIN, INPUT_PULLUP);
  pinMode(BUTTON2_PIN, INPUT_PULLUP);
  pinMode(RELAY1_PIN, OUTPUT);
  pinMode(RELAY2_PIN, OUTPUT);

  digitalWrite(RELAY1_PIN, LOW);
  digitalWrite(RELAY2_PIN, LOW);
}

void loop() {
  // Toggle relay 1 on button 1 press
  if (digitalRead(BUTTON1_PIN) == LOW) {
    delay(50);  // Debounce
    if (digitalRead(BUTTON1_PIN) == LOW) {
      relay1State = !relay1State;
      digitalWrite(RELAY1_PIN,␣
↪relay1State);
      while(digitalRead(BUTTON1_PIN) ==␣
↪LOW);  // Wait for release
    }
  }

  // Similar logic for relay 2
}
```

9.5.4 MicroPython Example

```
from machine import Pin
import time

# Initialize relay pins
relay1 = Pin(7, Pin.OUT)
relay2 = Pin(8, Pin.OUT)

# Turn relays off initially
relay1.off()
relay2.off()

while True:
    # Relay 1 on for 2 seconds
    relay1.on()
    time.sleep(2)
    relay1.off()
    time.sleep(1)

    # Relay 2 on for 2 seconds
    relay2.on()
    time.sleep(2)
    relay2.off()
    time.sleep(1)
```

9.6 Wiring Examples

9.6.1 Controlling AC Lamp (120V/220V)

```
AC Source (Hot) ——→ COM
                    NO ——→ Lamp ——→ AC␣
↪Source (Neutral)
                    NC ——→ (not connected)

When relay ON: Lamp receives power
When relay OFF: Lamp is disconnected
```

IMPORTANT SAFETY: Always consult a licensed electrician for AC wiring!

9.6.2 Controlling DC Motor

```
Battery (+) ——→ COM
                NO ——→ Motor (+)
                NC ——→ (not connected)
Motor (-) ——→ Battery (-)

When relay ON: Motor runs
When relay OFF: Motor stops
```

9.6.3 Reversing Motor Direction

Using both relays with H-bridge configuration:

```
Relay 1 controls motor positive connection
Relay 2 controls motor negative connection

Forward:  Relay1=ON,  Relay2=OFF
Reverse:  Relay1=OFF, Relay2=ON
Brake:    Relay1=OFF, Relay2=OFF
```

9.7 Safety Considerations

Electrical Safety

> **Warning:** Relays can switch dangerous voltages!
>
> - Never touch relay terminals when powered
> - Use appropriate wire gauge for current
> - Install proper fuses and circuit breakers
> - Follow local electrical codes
> - Consider hiring a licensed electrician for AC wiring

Load Considerations

- Do not exceed 10A contact rating
- Use inductive load suppression for motors
- Consider relay contact bounce
- Account for surge currents during switching
- Use appropriate wire insulation for voltage

Inductive Load Protection

For inductive loads (motors, solenoids, relays):

- Add flyback diode across DC loads

- Use snubber circuit for AC loads
- Prevents voltage spikes
- Protects relay contacts
- Extends relay lifetime

```
DC Motor Protection:
Motor --+--> Relay
        |
      Diode (1N4007 or similar)
        |
        v
       GND
```

9.8 Practical Applications

9.8.1 Home Automation

- Light control
- Fan control
- Appliance switching
- Irrigation systems
- Garage door control

9.8.2 Industrial Applications

- Motor control
- Pump control
- Valve actuation
- Heating element control
- Safety interlocks

9.8.3 IoT Projects

- Remote power control
- Scheduled device control
- Sensor-triggered switching
- Energy monitoring systems
- Smart home integration

9.8.4 Educational Projects

- Learning about relays and switching
- Building automation systems
- Understanding isolation
- Control system design
- Safety system implementation

9.9 Advanced Features

9.9.1 PWM Control

```
// Variable duty cycle control
void setup() {
  pinMode(RELAY1_PIN, OUTPUT);
}

void loop() {
  // 50% duty cycle (on 500ms, off 500ms)
  digitalWrite(RELAY1_PIN, HIGH);
  delay(500);
  digitalWrite(RELAY1_PIN, LOW);
  delay(500);
}
```

Applications: - Simple speed control for DC motors - Heater temperature regulation - Light dimming (not recommended for AC)

9.9.2 Interlock Logic

```
void setRelays(bool r1, bool r2) {
  // Ensure both relays are never on␣
↪simultaneously
  if (r1 && r2) {
    // Conflict - turn both off
    digitalWrite(RELAY1_PIN, LOW);
    digitalWrite(RELAY2_PIN, LOW);
    return;
  }

  digitalWrite(RELAY1_PIN, r1);
  digitalWrite(RELAY2_PIN, r2);
}
```

9.9.3 Failsafe Operation

```
unsigned long lastCommandTime = 0;
const unsigned long TIMEOUT = 5000;  // 5␣
↪seconds

void loop() {
  if (millis() - lastCommandTime >␣
↪TIMEOUT) {
    // No command received, turn off relays
    digitalWrite(RELAY1_PIN, LOW);
    digitalWrite(RELAY2_PIN, LOW);
  }
}
```

9.10 Troubleshooting

Relay Not Switching

- Verify control signal (should be HIGH/3.3V or 5V)
- Check power supply to relay module
- Test with multimeter on COM and NO/NC
- Listen for clicking sound when switching
- Check LED indicator

Relay Clicking but Not Switching Load

- Verify load voltage and current are within ratings
- Check wiring connections
- Test relay contacts with multimeter
- Look for burned or welded contacts
- Replace relay if damaged

Intermittent Operation

- Check loose wire connections
- Verify stable power supply
- Look for electromagnetic interference
- Check control signal integrity
- Ensure proper grounding

Relay Stays On

- Check control signal is going LOW
- Inspect for stuck contacts
- Verify relay is not damaged
- Check for control circuit faults
- Replace relay module if needed

9.11 Specifications Summary

Table 9.2: Operating Limits

Parameter	Maximum	Notes
AC Voltage	250V	Use proper insulation
DC Voltage	30V	Lower than AC rating
Current	10A	Continuous rating
Switching Frequency	300 ops/min	Mechanical limitation
Contact Resistance	<100 milliohms	When closed
Insulation Resistance	>100 Megaohms	When open
Operating Temperature	-10°C to +55°C	Typical range

CHAPTER
TEN

HOW TO GENERATE AN ERROR REPORT

This guide explains how to generate an error report using GitHub repositories.

10.1 Steps to Create an Error Report

1. **Access the GitHub Repository**
   Navigate to the GitHub repository where the project is hosted.
2. **Open the Issues Tab**
   Click on the "Issues" tab located in the repository menu.
3. **Create a New Issue**
   - Click the "New Issue" button.
   - Provide a clear and concise title for the issue.
   - Add a detailed description, including relevant information such as:
     - Steps to reproduce the error.
     - Expected and actual results.
     - Any related logs, screenshots, or files.
4. **Submit the Issue**
   Once the form is complete, click the "Submit" button.

10.2 Review and Follow-Up

The development team or maintainers will review the issue and take appropriate action to address it.